

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr. J. Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S. Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch. Rajitha	
		Mr. M Prakash	
		Mr. B. Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Tuesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 5.2 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 5: Ethical Foundations – Responsible AI Coding Practices Lab Objectives: - To explore the ethical risks associated with AI-generated code. - To recognize issues related to security, bias, transparency, and copyright. - To reflect on the responsibilities of developers when using AI tools in software development. - To promote awareness of best practices for responsible and ethical AI coding.	Week3 - Wednesday	

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.
- Understand the importance of explainability and transparency in AI-assisted programming.
- Reflect on accountability and the human role in ethical AI coding practices..

Task Description#1 (Privacy and Data Security)

- Use an AI tool (e.g., Copilot, Gemini, Cursor) to generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption.

Expected Output#1

- Identification of insecure logic; revised secure version with proper password hashing/encrypting and environment variable use.

VS CODE:

PROMT: Generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption.

```
Task1.py > ...
1  import hashlib
2  import getpass
3  import os
4  import json
5
6  # File to store user credentials securely (hashed passwords)
7  USER_DB_FILE = 'users_db.json'
8
9  def hash_password(password):
10     """Hash a password using SHA-256."""
11     return hashlib.sha256(password.encode()).hexdigest()
12
13  def load_users():
14     """Load users from the database file."""
15     if not os.path.exists(USER_DB_FILE):
16         return {}
17     with open(USER_DB_FILE, 'r') as f:
18         return json.load(f)
19
20  def save_users(users):
21     """Save users to the database file."""
22     with open(USER_DB_FILE, 'w') as f:
23         json.dump(users, f)
24
25  def register():
26     """Register a new user with username and password."""
27     users = load_users()
28     username = input("Enter new username: ")
29     if username in users:
30         print("Username already exists.")
31         return
32     password = getpass.getpass("Enter new password: ")
33     # Store only the hashed password, not plain-text
34     users[username] = hash_password(password)
35     save_users(users)
36     print("Registration successful.")
```

```

Task1.py > ...
37
38 def login():
39     """Login an existing user."""
40     users = load_users()
41     username = input("Enter username: ")
42     password = getpass.getpass("Enter password: ")
43     hashed = hash_password(password)
44     # Compare hashed password, not plain-text
45     if users.get(username) == hashed:
46         print("Login successful.")
47     else:
48         print("Invalid username or password.")
49
50 def main():
51     """Main function to choose register or login."""
52     while True:
53         print("\n1. Register\n2. Login\n3. Exit")
54         choice = input("Choose an option: ")
55         if choice == '1':
56             register()
57         elif choice == '2':
58             login()
59         elif choice == '3':
60             break
61         else:
62             print("Invalid choice.")
63
64 if __name__ == "__main__":
65     main()
66
67 # --- Security Review ---
68 # - No hardcoded passwords.
69 # - Passwords are not stored in plain-text; they are hashed using SHA-256.
70 # - No encryption for the user database file, but passwords are not readable.
71 # - For production, use salted hashes (e.g., bcrypt) and secure storage.

```

OUTPUT:

```

PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:\Users\supri\AppData\Local\Programs\Python\Python313\python.exe c:/Users/supri/OneDrive/Desktop/AIAC/Lab5/Task1.py

1. Register
2. Login
3. Exit
Choose an option: 1
Enter new username: SUPRIYA
Enter new password:
Registration successful.

1. Register
2. Login
3. Exit
Choose an option: 1
Enter new username: harsha
Enter new password:
Registration successful.

1. Register
2. Login
3. Exit
Choose an option: 3
PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5>

```

Task Description#2 (Bias)

- Use prompt variations like: “loan approval for John”, “loan approval for Priya”, etc. Evaluate whether the AI-generated logic exhibits bias or differing criteria based on names or genders.

Expected Output#2

- Screenshot or code comparison showing bias (if any); write 3–4 sentences on mitigation techniques.

PROMT: Generate a python code based on loan approval for john, loan approval for Priya and ect...evaluate whether the the AI-generated logic exhibits bias or differing criteria based on names or genders.

```

Task2.py > -
1 # loan approval logic for different applicants
2
3 def approve_loan(applicant):
4     # Example AI-generated logic (for demonstration purposes)
5     if applicant['name'].lower() == 'john':
6         return "Approved"
7     elif applicant['name'].lower() == 'priya':
8         return "Denied"
9     else:
10        # Generic logic based on income
11        if applicant['income'] > 50000:
12            return "Approved"
13        else:
14            return "Denied"
15
16 # Applicants
17 applicants = [
18     {'name': 'John', 'gender': 'Male', 'income': 40000},
19     {'name': 'Priya', 'gender': 'Female', 'income': 60000},
20     {'name': 'Alex', 'gender': 'Male', 'income': 70000},
21     {'name': 'Sara', 'gender': 'Female', 'income': 30000}
22 ]
23
24 # Evaluate loan approvals
25 for applicant in applicants:
26     result = approve_loan(applicant)
27     print(f"Applicant: {applicant['name']], Gender: {applicant['gender']], Income: {applicant['income']] -> Loan: {result}")
28
29 # Evaluation of bias:
30 print("\nEvaluation:")
31 print("The logic above approves or denies loans based on the applicant's name for 'John' and 'Priya', regardless of their income.")
32 print("This exhibits bias, as different criteria are applied based on the name, which may correlate with gender or ethnicity.")
33 print("For other applicants, the decision is based on income, which is a more objective criterion.")

```

Output:

```

PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:/Users/supri/AppData/Local/Programs/Python/Python313/python.exe c:/Users
Applicant: John, Gender: Male, Income: 40000 -> Loan: Approved
Applicant: Priya, Gender: Female, Income: 60000 -> Loan: Denied
Applicant: Alex, Gender: Male, Income: 70000 -> Loan: Approved
Applicant: Sara, Gender: Female, Income: 30000 -> Loan: Denied

Evaluation:
The logic above approves or denies loans based on the applicant's name for 'John' and 'Priya', regardless of their income.
This exhibits bias, as different criteria are applied based on the name, which may correlate with gender or ethnicity.
For other applicants, the decision is based on income, which is a more objective criterion.
PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5>

```

Task Description#3 (Transparency)

- Write prompt to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document

Expected Output#3

- Code with explanation
- **Assess: Is the explanation understandable and correct?**

PROMT: Generate a python code to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document.give me code with explanation and the explanation explanation understandable and correct

CODE:

```

Task3.py > ...
1 # Function to calculate the nth Fibonacci number using recursion
2 def fibonacci(n):
3     """
4     Recursively calculates the nth Fibonacci number.
5
6     The Fibonacci sequence is defined as:
7     F(0) = 0
8     F(1) = 1
9     F(n) = F(n-1) + F(n-2) for n > 1
10
11     Args:
12     | n (int): The position in the Fibonacci sequence (must be >= 0).
13
14     Returns:
15     | int: The nth Fibonacci number.
16     """
17     # Base cases: F(0) = 0, F(1) = 1
18     if n == 0:
19         return 0
20     elif n == 1:
21         return 1
22     else:
23         # Recursive case: F(n) = F(n-1) + F(n-2)
24         return fibonacci(n-1) + fibonacci(n-2)
25
26 # Example usage and output
27 if __name__ == "__main__":
28     # Get user input
29     n = int(input("Enter the position n to find the nth Fibonacci number: "))
30     result = fibonacci(n)
31     print(f"The {n}th Fibonacci number is: {result}")
32

```

```

Task3.py > ...
2 def fibonacci(n):
3     """
4     Recursively calculates the nth Fibonacci number.
5
6     The Fibonacci sequence is defined as:
7     F(0) = 0
8     F(1) = 1
9     F(n) = F(n-1) + F(n-2) for n > 1
10
11     Args:
12     | n (int): The position in the Fibonacci sequence (must be >= 0).
13
14     Returns:
15     | int: The nth Fibonacci number.
16     """
17     # Base cases: F(0) = 0, F(1) = 1
18     if n == 0:
19         return 0
20     elif n == 1:
21         return 1
22     else:
23         # Recursive case: F(n) = F(n-1) + F(n-2)
24         return fibonacci(n-1) + fibonacci(n-2)
25
26 # Example usage and output
27 if __name__ == "__main__":
28     # Get user input
29     n = int(input("Enter the position n to find the nth Fibonacci number: "))
30     result = fibonacci(n)
31     print(f"The {n}th Fibonacci number is: {result}")
32
33 """
34 Explanation:
35 - The function 'fibonacci' uses recursion to calculate the nth Fibonacci number.
36 - It checks for the base cases (n == 0 or n == 1) and returns the corresponding value.
37 - For n > 1, it calls itself with (n-1) and (n-2) and adds their results.
38 - The code includes a docstring explaining the function and its arguments.
39 - The main block allows the user to input a value for n and prints the result.
40
41 Assessment:
42 - The explanation is clear and correct.
43 - The code correctly implements the recursive Fibonacci calculation.
44 - For example, if the user inputs 5, the output will be:
45 | The 5th Fibonacci number is: 5
46 """
47

```

OUTPUT:

```

PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:/Users/supri/AppData/Local/Programs/python/python313/python.exe c:/Users/supri/OneDrive/Desktop/Task3.py
Enter the position n to find the nth Fibonacci number: 3
The 3th Fibonacci number is: 2
PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:/Users/supri/AppData/Local/Programs/Python/Python313/python.exe c:/Users/supri/OneDrive/Desktop/Task3.py
Enter the position n to find the nth Fibonacci number: 13
The 13th Fibonacci number is: 233
PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5>

```

Task Description#4 (Bias)

- Ask to generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings.

Expected Output#4

- Python code
- Analyze is there any bias with respect to gender or any

- **PROMT:** *Generate a python code of a job applicant scoring system based on input features features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings. And Analyze is there any bias with respect to gender or any*

CODE:

```
Task4.py > ...
1 # Job Applicant Scoring System
2
3 def score_applicant(education, experience, gender, age):
4     """
5     Scores a job applicant based on input features.
6     education: str ('highschool', 'bachelor', 'master', 'phd')
7     experience: int (years)
8     gender: str ('male', 'female', 'other')
9     age: int
10    Returns: int (score)
11    """
12    # Education scoring
13    edu_scores = {
14        'highschool': 10,
15        'bachelor': 20,
16        'master': 30,
17        'phd': 40
18    }
19    score = edu_scores.get(education.lower(), 0)
20
21    # Experience scoring
22    if experience < 2:
23        score += 5
24    elif experience < 5:
25        score += 15
26    else:
27        score += 25
28
29    # Age scoring (neutral, just for demonstration)
30    if 18 <= age <= 60:
31        score += 10
32    else:
33        score += 0
34
35    # Gender scoring (neutral, no bias)
36    # No points added or subtracted for gender
37
38    return score
39
40 def analyze_bias(education, experience, gender, age):
41     """
42     Analyzes if there is any bias in the scoring logic.
43     """
44     print("Scoring logic does not use gender in calculation, so no gender bias.")
45     print("Education and experience are weighted based on typical job requirements.")
46     print("Age is only used to ensure working age, not to favor any group.")
47     print("If you input different genders, the score will remain the same for identical education, experience, and age.")
48
49 # Example usage
50 if __name__ == "__main__":
51     # Input from user
52     education = input("Enter education level (highschool/bachelor/master/phd): ")
53     experience = int(input("Enter years of experience: "))
54     gender = input("Enter gender (male/female/other): ")
55     age = int(input("Enter age: "))
56
57     score = score_applicant(education, experience, gender, age)
58     print(f"Applicant Score: {score}")
59
60     analyze_bias(education, experience, gender, age)
```

OUTPUT:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Scoring logic does not use gender in calculation, so no gender bias.
PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:\Users\supri\AppData\Local\Programs\Python\Python313\python.exe c:/Users/supri/OneDrive
Enter education level (highschool/bachelor/master/phd): phd
Enter years of experience: 5
Enter gender (male/female/other): male
Enter age: 35
Applicant Score: 75
Scoring logic does not use gender in calculation, so no gender bias.
Enter education level (highschool/bachelor/master/phd): phd
Enter years of experience: 5
Enter gender (male/female/other): male
Enter age: 35
Applicant Score: 75
Scoring logic does not use gender in calculation, so no gender bias.
Enter years of experience: 5
Enter gender (male/female/other): male
Enter age: 35
Applicant Score: 75
Scoring logic does not use gender in calculation, so no gender bias.
Enter gender (male/female/other): male
Enter age: 35
Applicant Score: 75
Scoring logic does not use gender in calculation, so no gender bias.
Education and experience are weighted based on typical job requirements.
Applicant Score: 75
Scoring logic does not use gender in calculation, so no gender bias.
Education and experience are weighted based on typical job requirements.
Scoring logic does not use gender in calculation, so no gender bias.
Education and experience are weighted based on typical job requirements.
Age is only used to ensure working age, not to favor any group.
Education and experience are weighted based on typical job requirements.
Age is only used to ensure working age, not to favor any group.
Age is only used to ensure working age, not to favor any group.
If you input different genders, the score will remain the same for identical education, experience, and age.
PS C:\Users\supri\OneDrive\Desktop\AIAC\Lab5>

```

Task Description#5 (Inclusiveness)

- Code Snippet

```

def greet_user(name, gender):
    if gender.lower() == "male":
        title = "Mr."
    else:
        title = "Mrs."
    return f"Hello, {title} {name}! Welcome."

```

Expected Output#5

- Regenerate code that includes **gender-neutral** also

```

Task5.py > ...
1  def greet_user(name, gender):
2      if gender.lower() == "male":
3          title = "Mr."
4      else:
5          title = "Mrs."
6      return f"Hello, {title} {name}! Welcome."
7
8
9  # Example usage:
10 if __name__ == "__main__":
11     name = input("Enter your name: ")
12     gender = input("Enter your gender (male/female): ")
13

```

```
S C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:/Users/supri/AppData/Local/Pr
Enter your name: supriya
Enter your gender (male/female): female
S C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> & C:/Users/supri/AppData/Local/Pr
Enter your name: raju
Enter your gender (male/female): male
S C:\Users\supri\OneDrive\Desktop\AIAC\Lab5> █
```

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Transparency	0.5
Bias	1.0
Inclusiveness	0.5
Data security and Privacy	0.5
Total	2.5 Marks